Modern NETtalk

Andrew, Dylan, Atul, Henry

ROSE-HULMAN INSTITUTE OF TECHNOLOGY

Why It Matters: Understanding Speech from Text

Purpose: Explore how machines learn to pronounce words

Problem: Mapping letters (graphemes) to sounds (phonemes)

Importance: speech synthesis enables accessibility and communication

Challenge: English spelling inconsistencies and complex sound patterns

Approach: Data-driven model trained on the CMU Pronouncing Dictionary

CMU Dataset (Carnegie Mellon Pronouncing Dictionary)

- 135,166 total Words → ARPAbet 2-word Phonemes
- Free to use for any purpose
- Don’t guarantee accuracy
- Actively curated by Dr. Alex Rudnicky

cmudict.dict	10 months ago
cmudict.phones	12 years ago
cmudict.symbols	11 years ago

CMUdict: Overview

- ~~84~~ 39 distinct phonemes are in the CMUdict
- Examples:
  - beer → B IH1 R
  - beer's → B IY1 R Z
  - beerbower → B IH1 R B OW0 ER0
  - beerman → B IH1 R M AH0 N
  - beers → B IH1 R Z
  - beers' → B IH1 R Z
  - beers's → B IH1 R Z IH0 Z
  - beery → B IH1 R IY0

Vowels[2]

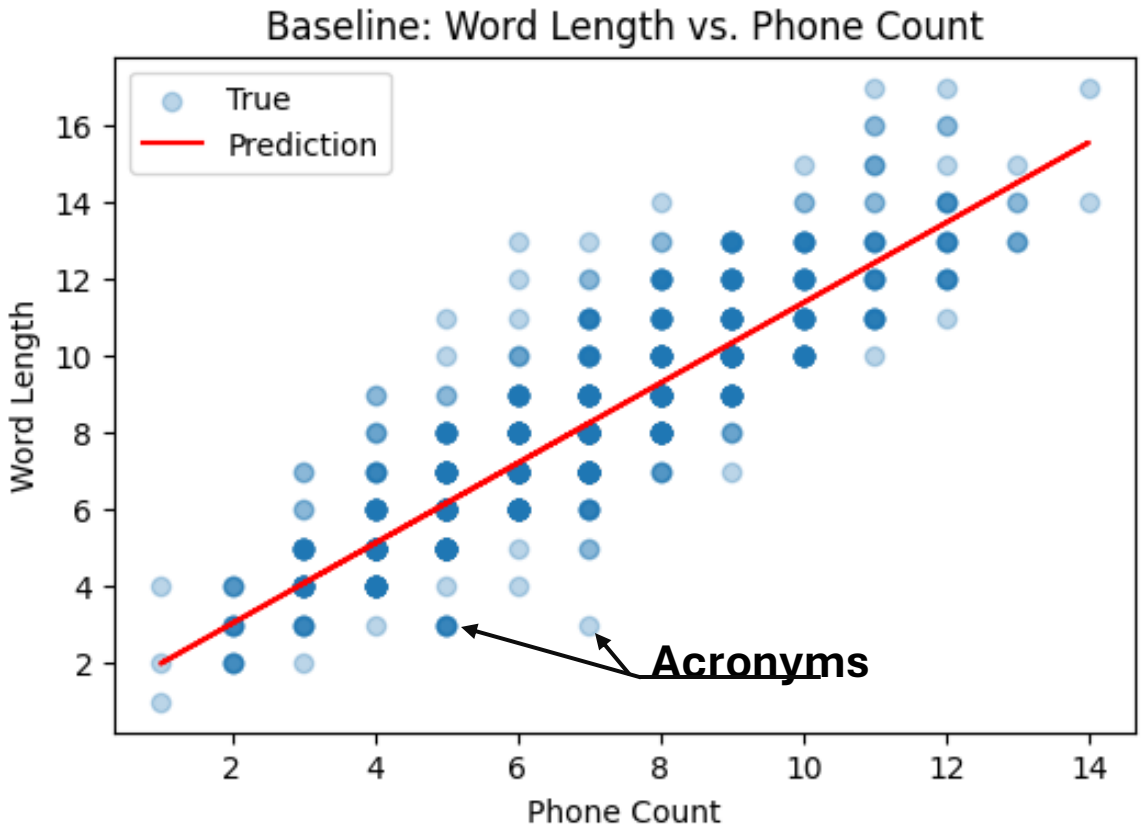
ARPABET		IPA	Example(s)
1-letter	2-letter		
a	AA	ɑ~ɒ	balm, bot (with father–bother merger)
@	AE	æ	bat
A	AH	ʌ	butt
c	AO	ɔ	caught, story
W	AW	aʊ	bout
x	AX	ə	comma
—	AXR[3]	ɚ	letter, forward
Y	AY	aɪ	bite

ROSE-HULMAN INSTITUTE OF TECHNOLOGY

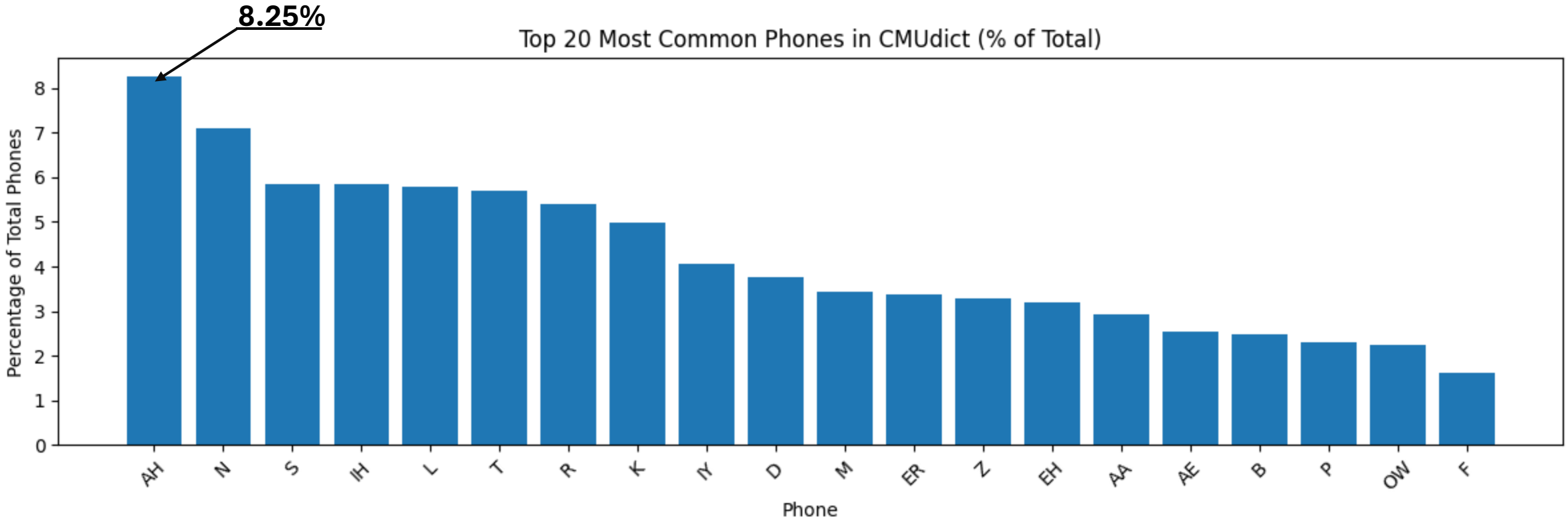
Dataset

- aaa → T R IH2 P AH0 L EY1
- bbq → B IY1 B IY0 K Y UW2
- etc → EH2 T S EH1 T ER0 AH0
- ltd → L IH1 M IH0 T IH0 D
- xml → EH2 K S EH2 M EH1 L

- Linear Fit: R^2 = 0.77

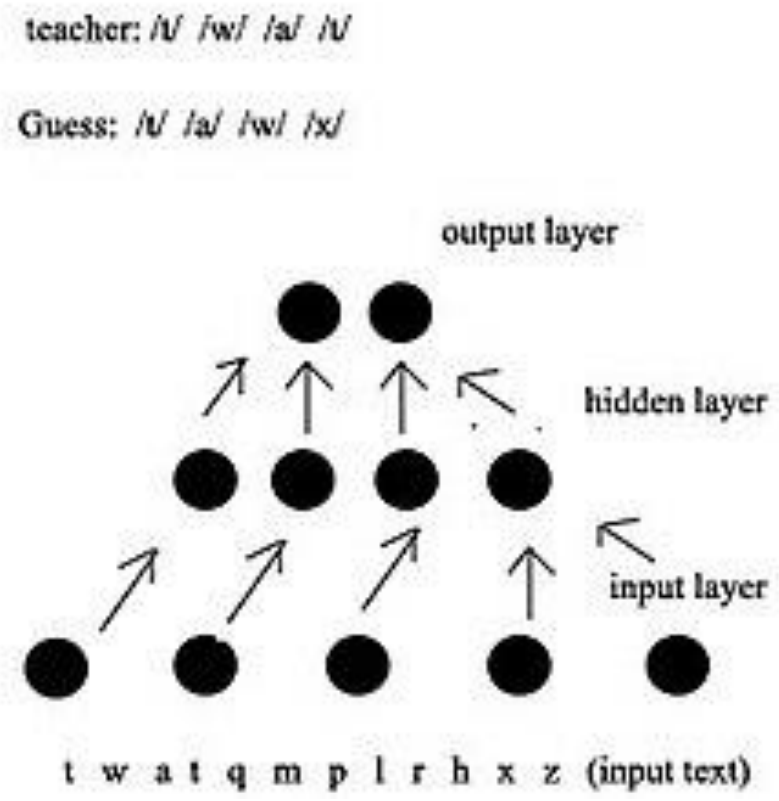


Baseline Regressor



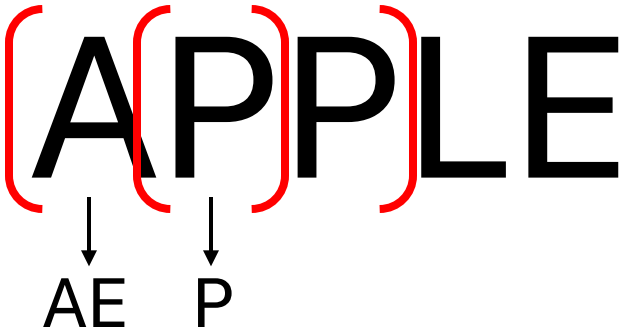
Quick Recap: NETTalk

- 1980s
- 20,008 words → Custom Phoneme Translation
- Input → Hidden Layer → Output
- DECtalk synthesizer
- ~3 months → 250 hours dataset creation → few days training



Technical Difficulties – Repeated Phonemes

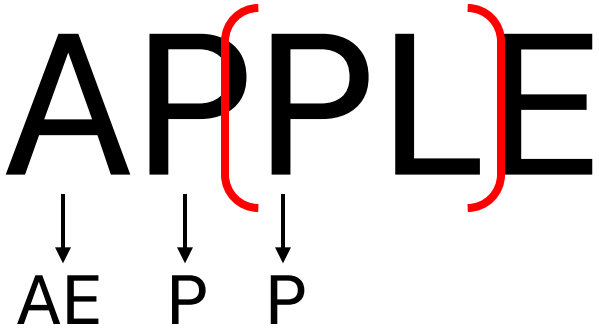
- Baseline Model Output
  - Sanburn: S AE1 N B ER0 N N → S AE1 N B ER0 N
  - Serj: S ER1 JH JH → S ER1 JH
- Caused due to inherent architectural choices initially made



ROSE-HULMAN INSTITUTE OF TECHNOLOGY

Technical Difficulties – Repeated Phonemes

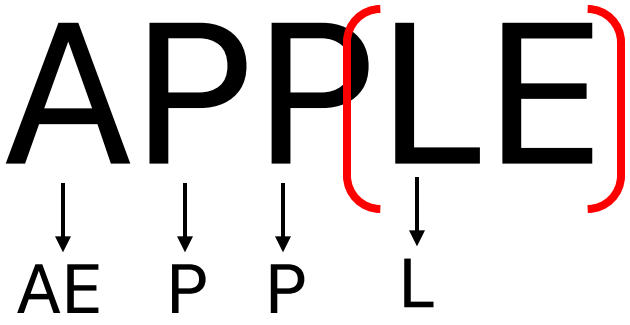
- Baseline Model Output
  - Sanburn: S AE1 N B ER0 N N → S AE1 N B ER0 N
  - Serj: S ER1 JH JH → S ER1 JH
- Caused due to inherent architectural choices initially made



ROSE-HULMAN INSTITUTE OF TECHNOLOGY

Technical Difficulties – Repeated Phonemes

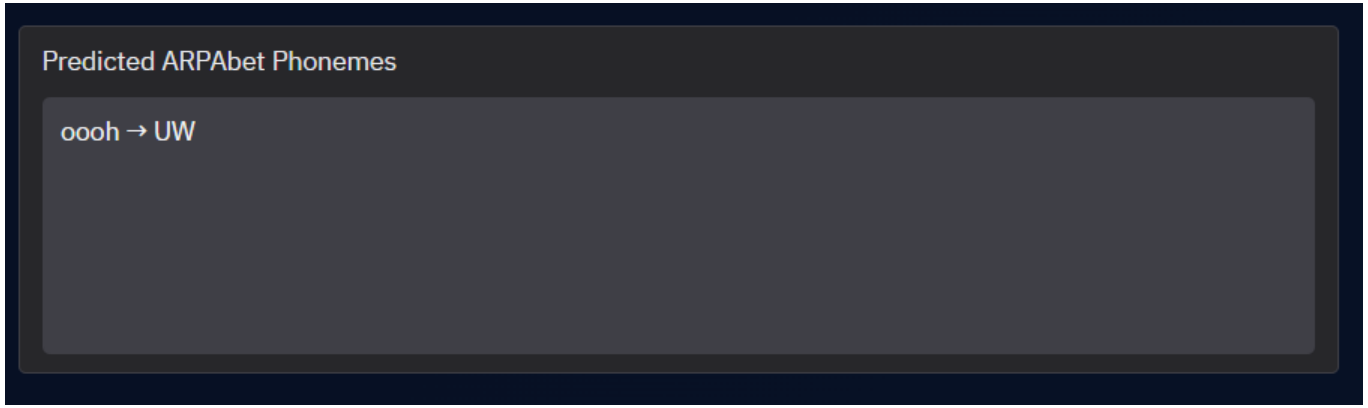
- Baseline Model Output
  - Sanburn: S AE1 N B ER0 N N → S AE1 N B ER0 N
  - Serj: S ER1 JH JH → S ER1 JH
- Caused due to inherent architectural choices initially made



ROSE-HULMAN INSTITUTE OF TECHNOLOGY

Technical Difficulties – Non-Linear Mapping

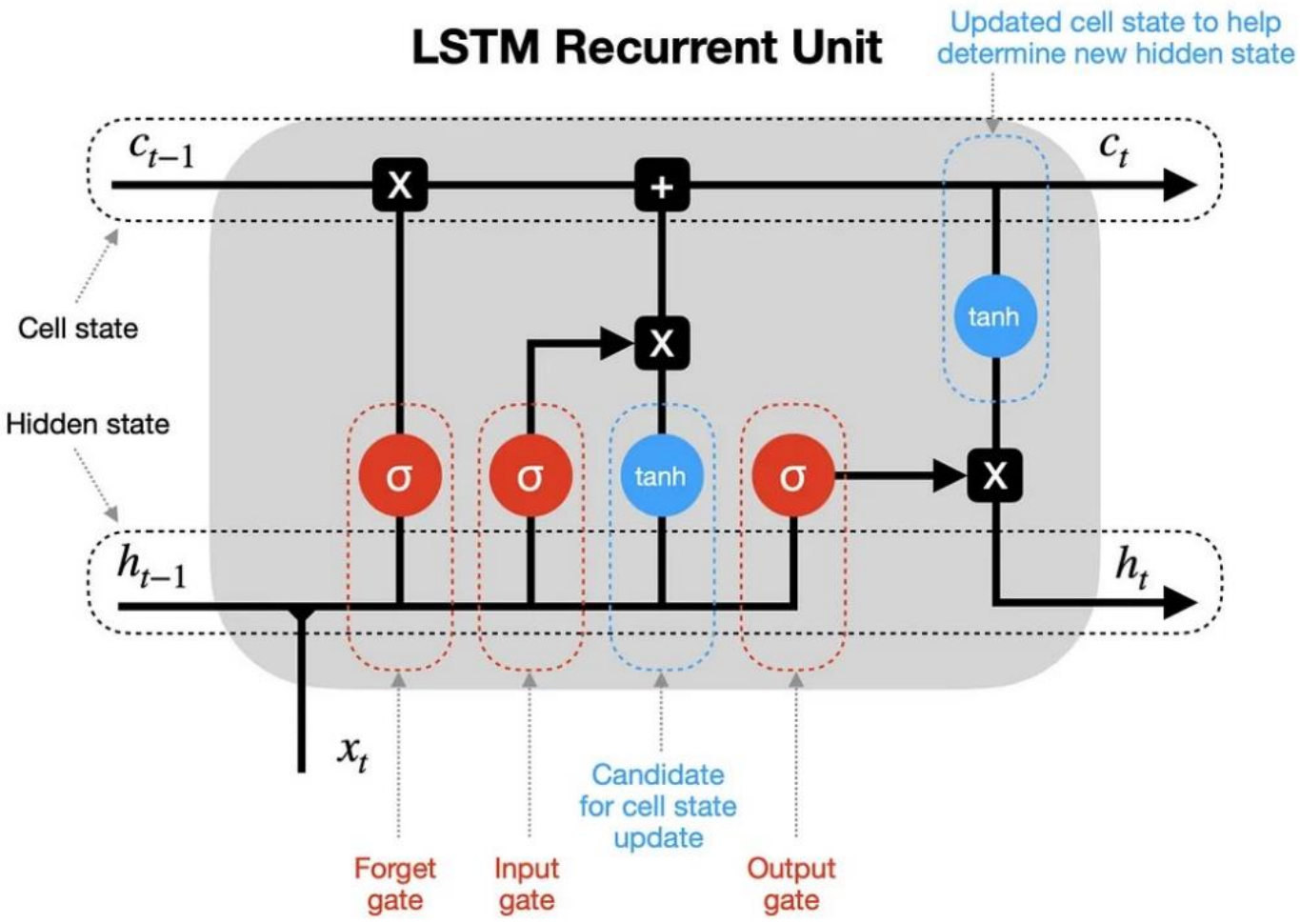
- More Outputs than Inputs
  - tough: T AH1 F AH0 Z → T AH1 F
  - cough: K AW1 F AH0 Z → K AO1 F or K AA1 F
  - The initial model would add extra phonemes to match the number of letters
  - The LSTM model works through the words with a sliding linear window, it can't generate a word with more phonemes than letters



Old iteration of website

What is LSTM?!?!

- PyTorch LSTM
- Made to solve the Vanishing Gradient Problem in RNNs
- Process sequential data (Natural Language Processing)
- One way or Bi-Directional



OUR LSTM Model

Model Configuration

Embedding Dim
64

LSTM Layers
2

Dropout
0.1

Learning Rate
1e-3

Hidden Dim
256

Bidirectional
Yes

Batch Size
32

Epochs
15

LSTM

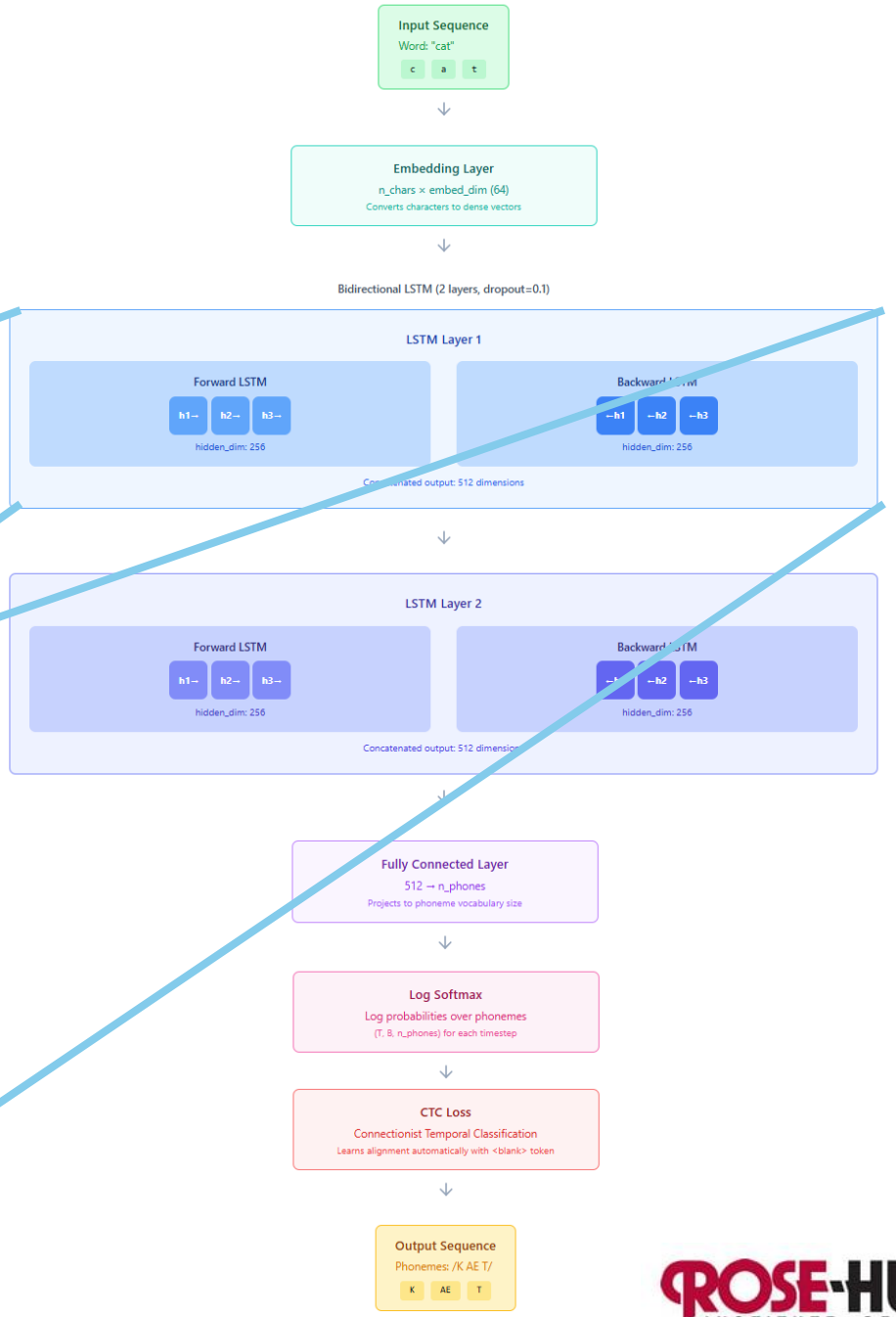
class torch.nn.LSTM(*input_size, hidden_size, num_layers=1, bias=True, batch_first=False, dropout=0.0, bidirectional=False, proj_size=0, device=None, dtype=None*) [source]

Apply a multi-layer long short-term memory (LSTM) RNN to an input sequence. For each element in the input sequence, each layer computes the following function:

$$
\begin{aligned}
i_t &= \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\
f_t &= \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\
g_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\
o_t &= \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}
$$

where h_t is the hidden state at time *t*, c_t is the cell state at time *t*, x_t is the input at time *t*, h_{t-1} is the hidden state of the layer at time *t-1* or the initial hidden state at time *0*, and i_t, f_t, g_t, o_t are the input, forget, cell, and output gates, respectively. σ is the sigmoid function, and $\odot$ is the Hadamard product.

In a multilayer LSTM, the input $x_t^{(l)}$ of the l -th layer ($l \geq 2$) is the hidden state $h_t^{(l-1)}$ of the previous layer multiplied by dropout $\delta_t^{(l-1)}$ where each $\delta_t^{(l-1)}$ is a Bernoulli random variable which is 0 with probability `dropout`.



ROSE-HULMAN INSTITUTE OF TECHNOLOGY

OUR LSTM Model

Grapheme-to-Phoneme (G2P) Experimental Setup

Dataset

CMU Pronouncing Dictionary

- ~135,000 word → phoneme pairs
- 84 phoneme classes
- Stress markers removed

(AE1 → AE)

Split:
- 80% Training
- 20% Testing

→

Preprocessing

- Words → character indices
- Phonemes → phoneme indices
- CTC-compatible formatting

Constraint:
word_length ≥ phoneme_length

→

Model Training

Bidirectional LSTM + CTC

- 15 epochs
- Optimizer: AdamW
- LR scaled by batch size
- Batch size: 64

```
Word: google
Predicted: G UW G AH L
Reference: G UW G AH L
Phoneme Error Rate (PER): 0.0%

Word: physics
Predicted: F IH Z IH K S
Reference: F IH Z IH K S
Phoneme Error Rate (PER): 0.0%

Word: algorithm
Predicted: AE L G ER IH TH AH M
Reference: AE L G ER IH DH AH M
Phoneme Error Rate (PER): 12.5%

Word: data
Predicted: D AE T AH
Reference: D AE T AH
Phoneme Error Rate (PER): 0.0%

Word: chatgpt
Predicted: CH AE T JH P T
Reference: <missing>
```

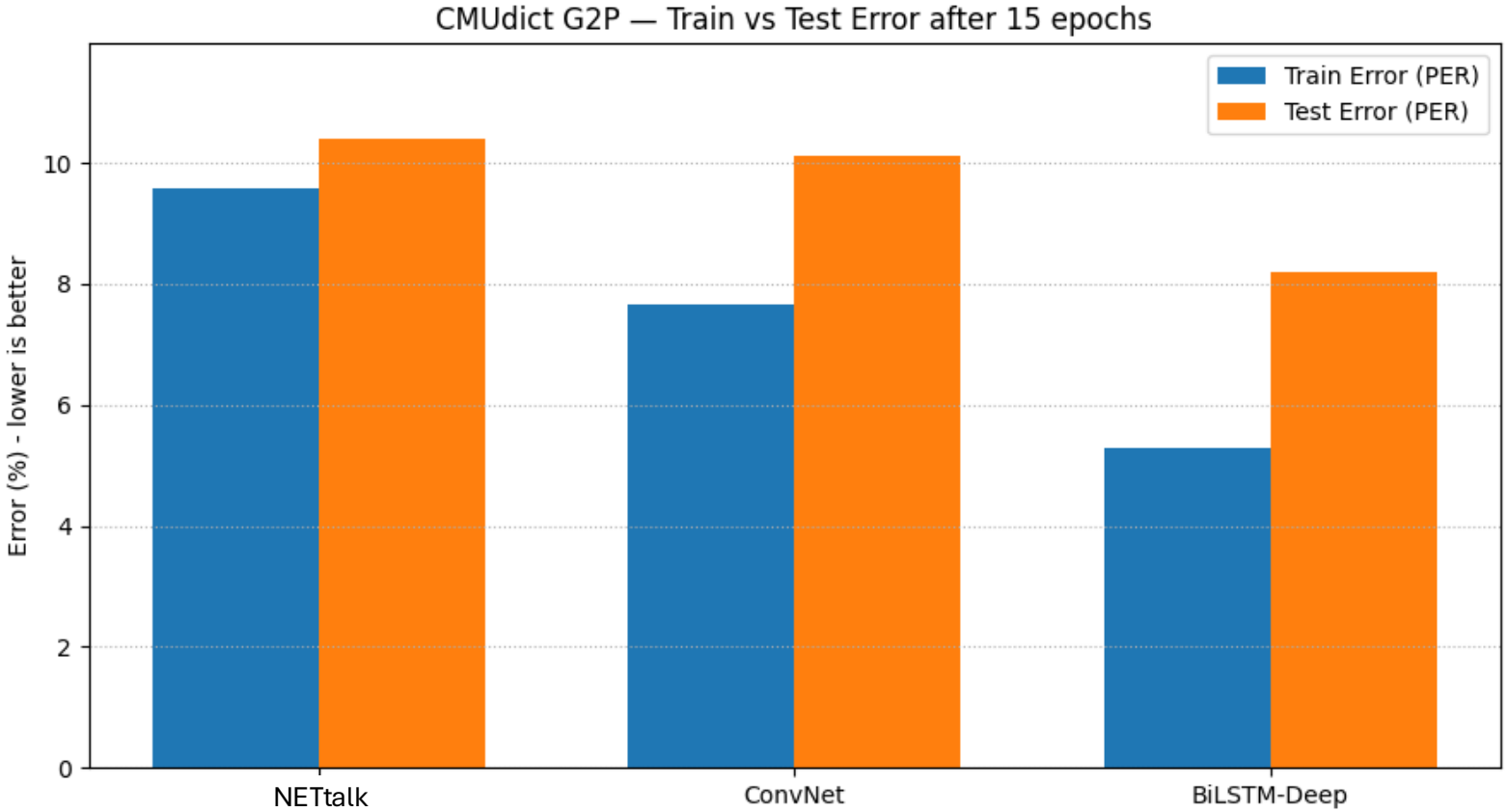
Improved Model Architecture

ROSE-HULMAN INSTITUTE OF TECHNOLOGY

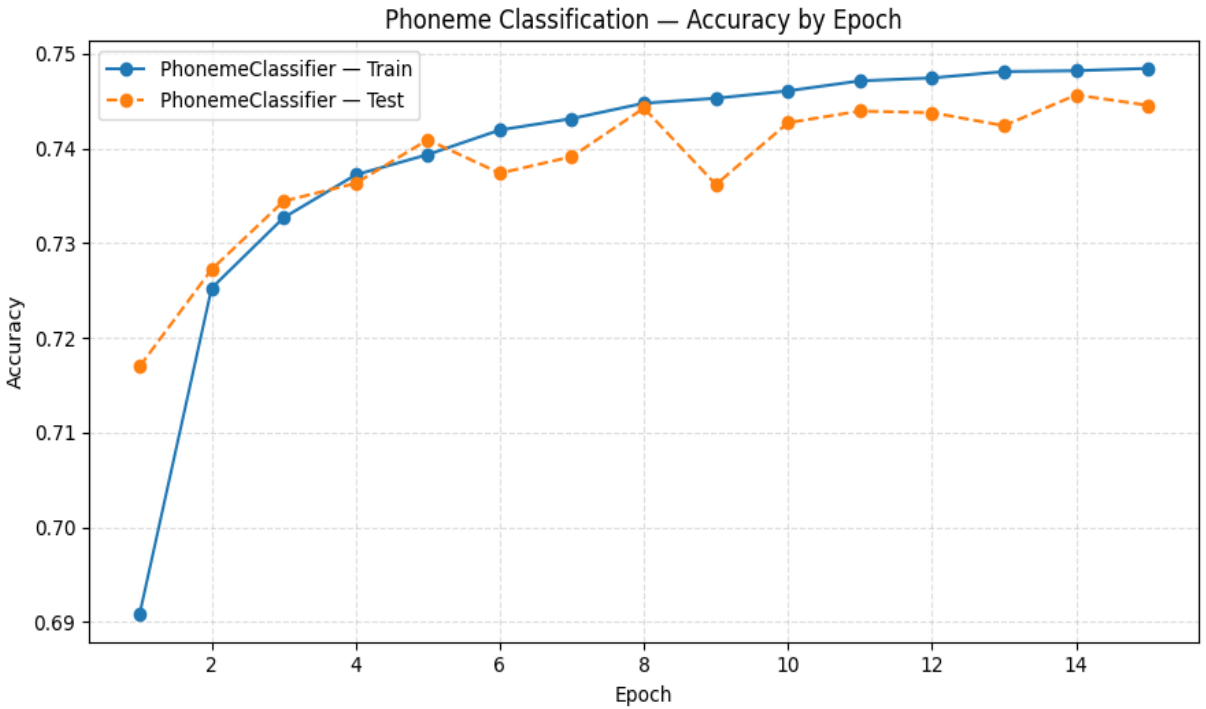
Model Architecture

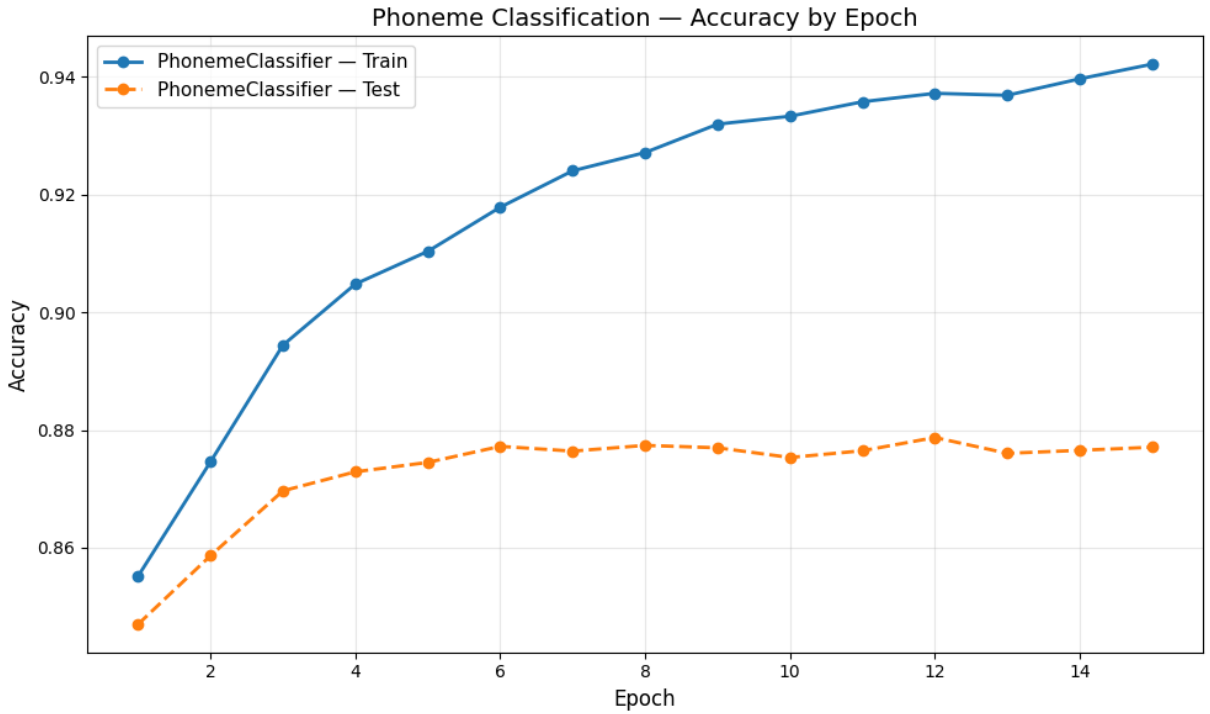
We had 3 iterations of our models

- 2-Layer Network (NETtalk design)
- CNN to 2-layer Network
- LSTM model



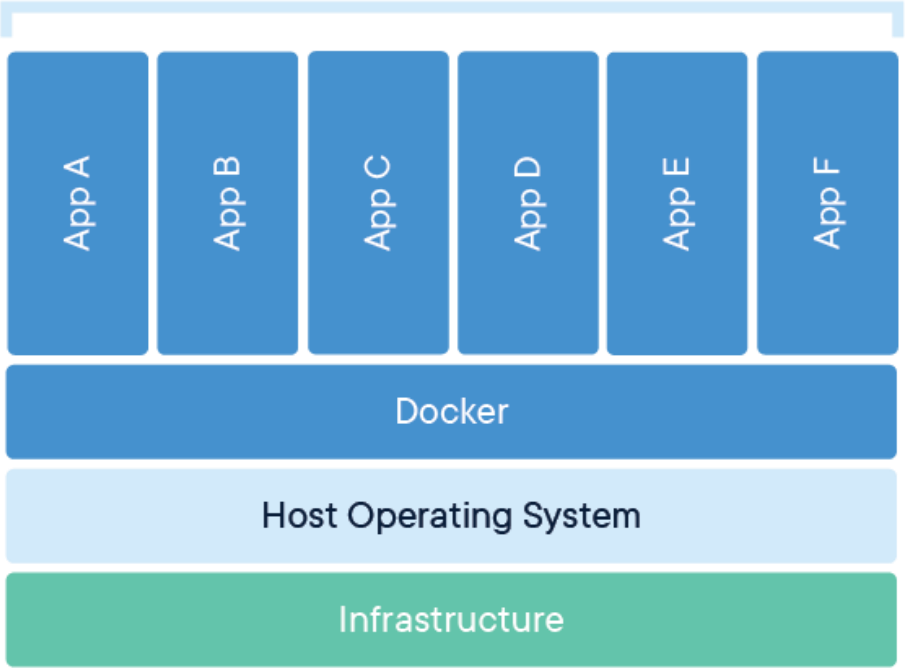
NETTalk architecture Vs. OUR architecture

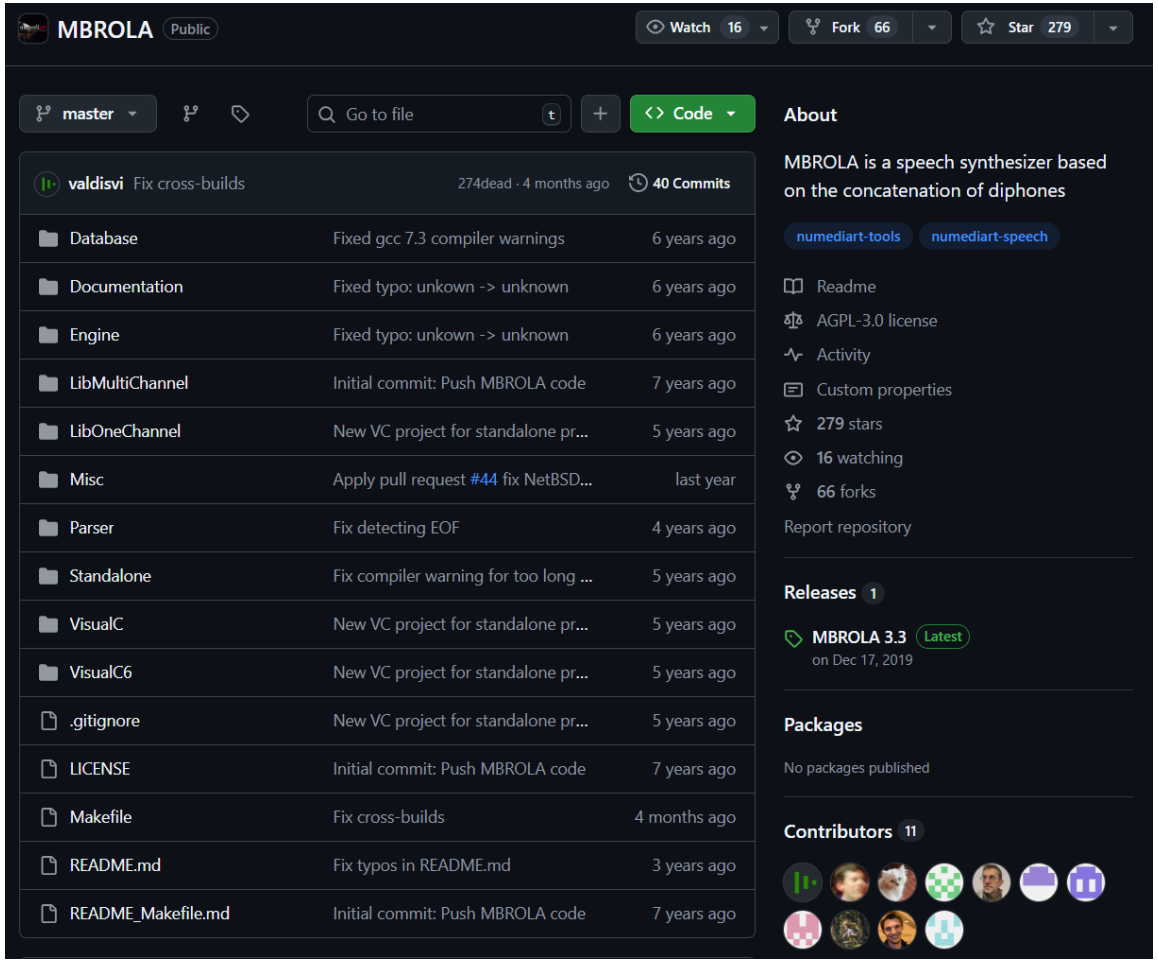




Technical Difficulties – Synthesizer

SAMPA	IPA	Description	Examples
p	p	voiceless bilabial plosive	English ***pen***, Croatian ***pas*** 'dog'
b	b	voiced bilabial plosive	English ***but***, Polish ***brat*** 'brother'
t	t	voiceless alveolar plosive	English ***two***, Spanish ***tomar*** 'to take'
d	d	voiced alveolar plosive	English ***do***, Polish ***dom*** 'house'
ts	ts	voiceless alveolar affricate	Italian ***calza*** 'sock', German ***Zeit*** 'time'
dz	dz	voiced alveolar affricate	Italian ***zona*** 'zone', Canadian French ***dur*** 'hard'
tS	tʃ	voiceless postalveolar affricate	English ***chair***, Spanish ***mucho*** 'many'





MBROLA Synthesizer

Mapping from CMU/Arpabet to SAMPA

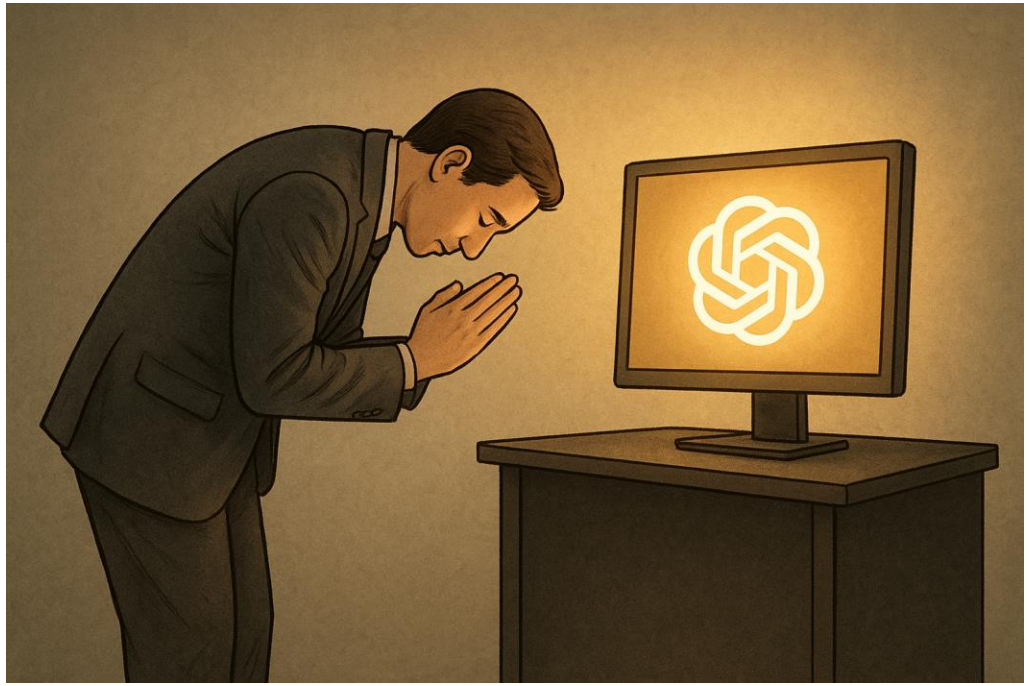
CMU/Arpabet	SAMPA	Notes
AA	A:	"balm" → barn sound
AE	{	"bat" → pat sound
AH	V	"butt" → putt sound
AO	O:	"caught" → born sound
AW	aU	"bout" → now sound
AY	aI	"bite" → buy sound
B	b	"buy" → but sound
CH	tS	"China" → chain sound
D	d	"die" → den sound
DH	D	"thy" → then sound
EH	e	"bet" → pet sound
ER	3:	"bird" → burn sound
Y	j	"yacht" → yes sound
Z	z	"zoo" → zeal sound
ZH	Z	"pleasure" → measure sound (exact match)

ROSE-HULMAN INSTITUTE OF TECHNOLOGY

DEMO

What would you change with more time?

- Use a transformer based-architecture
- Figure out a way to predict more phonemes than letters in word
- Get full model with synthesizer working on hosted webpage
  - Currently only predictive model
  - Must run locally
- Does not work with numerals



Questions?

Core References

Original NETtalk paper:

Parallel Networks that Learn to Pronounce English Text

Grapheme-to-Phoneme Conversion with Convolutional Neural Networks :
https://www.mdpi.com/2076-3417/9/6/1143?type=check_update&version=1

Grapheme-To-Phoneme Conversion Using Long Short-Term Memory Recurrent Neural Networks:

https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/43264.pdf

God(of computers):

https://www.chatgpt.com